Mehrdimensionale Arrays

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Referenzen in Java
- Speicherverwaltung mit Stack und Heap

Was wird in diesem Foliensatz besprochen?

- Mehrdimensionale Arrays in Java
- Kopieren von komplexeren Strukturen

Was lernen Sie kennen?

- Struktur von mehrdimensionalen Arrays
- Verwendung von mehrdimensionalen Arrays
- Kopieren von mehrdimensionalen Arrays

Mehrdimensionale Arrays

Arrays können auch mehrere Dimensionen haben

Beispiel: 2-dimensionales (2D) Array (z.B. Matrix)

```
int[][] matrix = new int[3][3];
int[][] matrix2 = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
```

Indexwerte

```
int[][] a = new int[3][4];
```

Zeile 0

[0, 0]	[0, 1]	[0, 2]	[0, 3]
[1, 0]	[1, 1]	[1, 2]	[1, 3]
[2, 0]	[2, 1]	[2, 2]	[2, 3]

Spalte 1

Konzeptionelle Sicht auf Arrays

```
int[][] a = new int[3][4];
```

a →

0	0	0	0
0	0	0	0
0	0	0	0

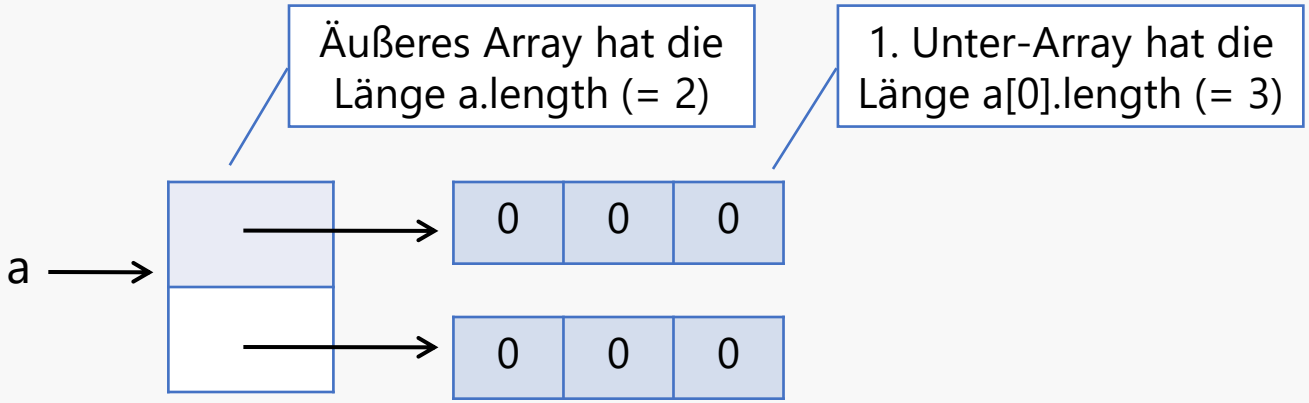
a ist eine Referenz auf ein zweidimensionales Array

Zugriff auf einzelnes Element

- Beispiel: x = a[2][3];

Realer Aufbau in Java (Array von Arrays)

```
int[][] a = new int[2][3];
```



Eindimensionale Arrays enthalten Elemente vom Basistyp
Mehrdimensionale Arrays enthalten weitere Arrays

Beispiel (2D-Array erzeugen und ausgeben)

```java
public class Array2D {
    public static void main(String[] args) {
        int[][] a = new int[2][3];
        for (int i = 0; i < a.length; i++) {
            for (int j = 0; j < a[i].length; j++) {
                a[i][j] = (int) (Math.random() * 10);
            }
        }
        for (int i = 0; i < a.length; i++) {
            for (int j = 0; j < a[i].length; j++) {
                System.out.print(a[i][j] + " ");
            }
            System.out.println();
        }
        for (int[] i : a) {
            for (int j : i) {
                System.out.print(j + " ");
            }
            System.out.println();
        }
    }
}
```

Ausgaben mit unterschiedlichen for-Varianten

```
7 8 3
4 2 7
7 8 3
4 2 7
```

Beispiel (2D-Array mit Zahlenmuster)

```java
public class Array2DPattern {

    public static void main(String[] args) {
        int n = 3;
        int[][] array = new int[n][n + 2];
        int counter = 1;
        for (int i = 0; i < array.length; i++) {
            for (int j = 0; j < array[i].length; j++) {
                array[i][j] = counter++ % n;
            }
        }

        for (int i = 0; i < array.length; i++) {
            for (int j = 0; j < array[i].length; j++) {
                if (j != 0) {
                    System.out.print("\t");
                }
                System.out.print(array[i][j]);
            }
            System.out.println();
        }
    }

}
```

Mehr Spalten als Zeilen, abwechselndes Zahlenmuster über Zeilen hinweg

```
1	2	0	1	2
0	1	2	0	1
2	0	1	2	0
```

Beispiel (2D-Array mit Zeichenmuster)

```java
public class Aray2DCharPattern {
    public static void main(String[] args) {
        int n = 11;
        char[][] array = new char[n][n];
        for (int i = 0; i < array.length; i++) {
            for (int j = 0; j < array[i].length; j++) {
                int distance = (Math.abs(n / 2 - i) + Math.abs(n / 2 - j));
                if (distance < n / 4) {
                    array[i][j] = ' ';
                } else if (distance < n / 2) {
                    array[i][j] = '.';
                } else {
                    array[i][j] = '#';
                }
            }
        }
        for (int i = 0; i < array.length; i++) {
            for (int j = 0; j < array[i].length; j++) {
                System.out.print(array[i][j]);
            }
            System.out.println();
        }
    }
}
```

Abhängig von der Distanz zur Mitte wird ein entsprechendes Zeichen gespeichert

```
###########
#####.#####
####...####
###.....###
##...  ...##
#...    ...#
##...  ...##
###.....###
####...####
#####.#####
###########
```

Weitere Aspekte (1)

Einzelne Dimensionen können offen bleiben (nur am Ende) und später initialisiert werden

```
int[][] arr1 = new int[2][];
for (int i = 0; i < arr1.length; i++)
    arr1[i] = new int [5];
```

Weitere Aspekte (2)

Einzelne Dimensionen können auch unterschiedliche Längen haben

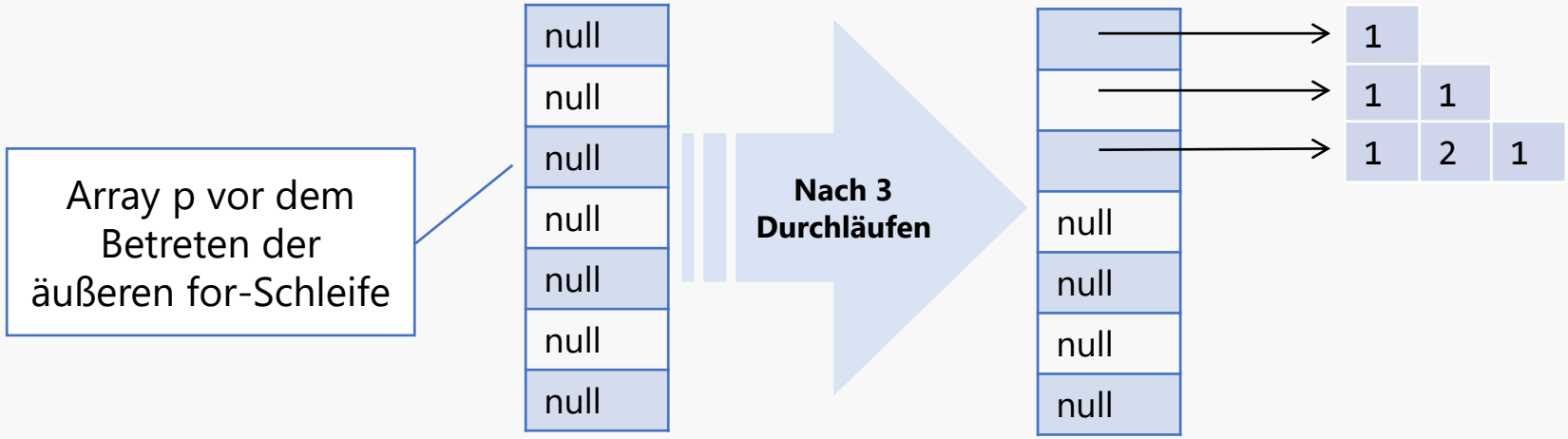
```
int[][] arr1 = new int[2][];
for (int i = 0; i < arr1.length; i++)
    arr1[i] = new int[i + 2];
```

Beispiel (Pascalsches Dreieck)

```java
import java.util.Scanner;

public class PascalTriangle {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Please enter a positive number: ");
        int n = sc.nextInt();
        int[][] p = new int[n][];
        for (int i = 0; i < p.length; i++) {
            p[i] = new int[i + 1];
            for (int j = 0; j <= i; j++) {
                if ((j == 0) || (j == i)) {
                    p[i][j] = 1;
                } else {
                    p[i][j] = p[i - 1][j - 1] + p[i - 1][j];
                }
                System.out.printf("%2d ", p[i][j]);
            }
            System.out.println();
        }
    }

}
```

Beispiel (Pascalsches Dreieck, 7 Stufen)

```
Please enter a positive number: 7
1
1  1
1  2  1
1  3  3  1
1  4  6  4  1
1  5 10 10  5  1
1  6 15 20 15  6  1
```



Beispiel (verschieden lange Zeilen)

```java
public class JaggedArrayTest {

    public static void main(String[] args) {
        int n = 5;
        int[][] testArray = new int[n][];
        int counter = 0;
        for (int i = 0; i < testArray.length; i++) {
            testArray[i] = new int[(int) (Math.random() * n)];
            for (int j = 0; j < testArray[i].length; j++) {
                testArray[i][j] = counter++;
            }
        }
        for (int[] line : testArray) {
            for (int number : line) {
                System.out.print(number + "\t");
            }
            System.out.println();
        }
    }
}
```

```
0    1
2    3    4
5    6    7    8
9    10
11   12   13
```

```
0
1    2    3
4
5    6
7    8    9
```

Belegung (und Länge der Zeilen) ändert sich bei jeder Ausführung

Flache vs. tiefe Kopie (clone bei 2D-Arrays)

Bei einem 2D-Array wird beim Aufruf von clone nur die erste Ebene kopiert (Referenzen auf weitere Arrays)!

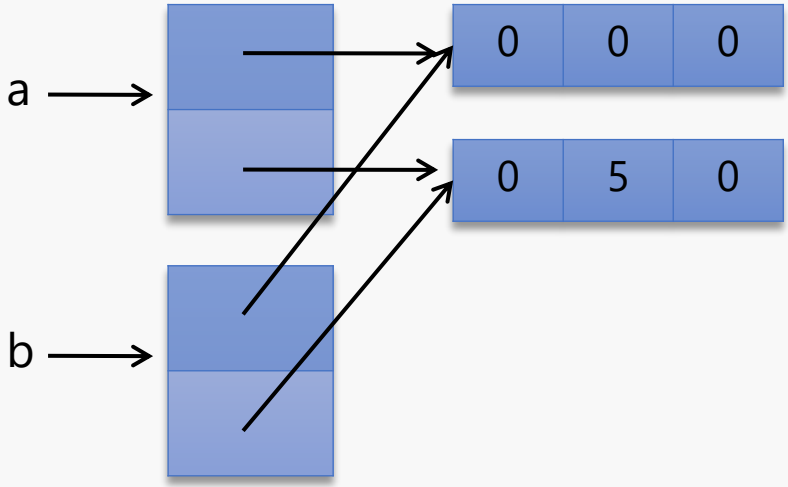
Hier spricht man von einer **flachen** Kopie

Bei einer **tiefen** Kopie müssen **alle** Ebenen kopiert werden

Flache Kopie mit clone

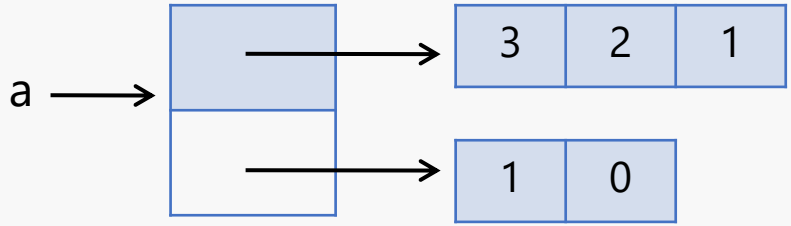
```
int[][] a = new int[2][3];
int[][] b = a.clone();
b[1][1] = 5;
System.out.println(a[1][1]);
System.out.println(b[1][1]);
```

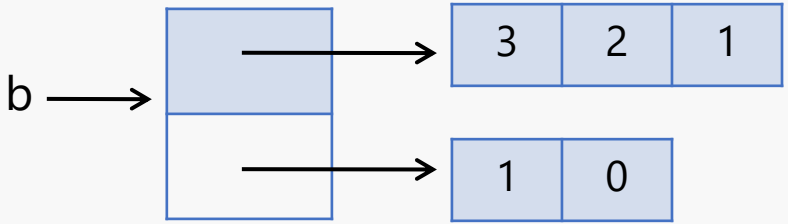
```
5
5
```

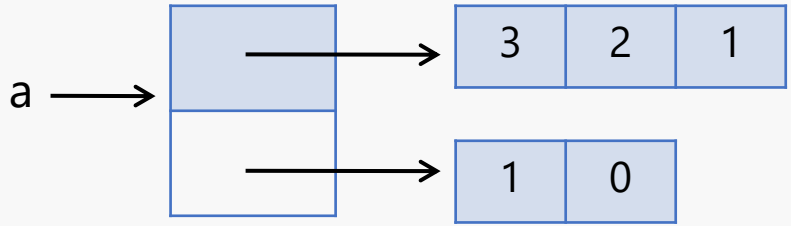
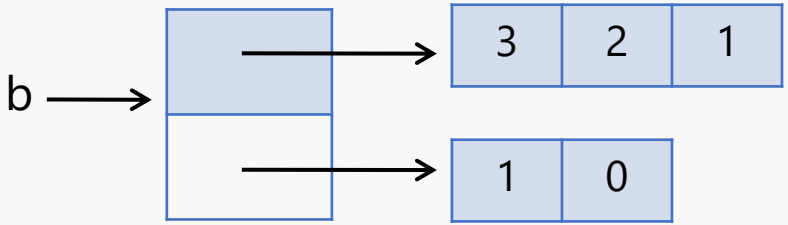


Tiefe Kopie

```
int[][] a = new int[][]{{3, 2, 1}, {1, 0}};
int[][] b = new int[a.length][];
for (int i = 0; i < a.length; i++) {
    b[i] = new int[a[i].length];
    for (int j = 0; j < a[i].length; j++) {
        b[i][j] = a[i][j];
    }
}
```





Tiefe Kopie mit System.arraycopy

```
int[][] a = new int[][]{{3, 2, 1}, {1, 0}};
int[][] b = new int[a.length][];
for (int i = 0; i < a.length; i++) {
    b[i] = new int[a[i].length];
    System.arraycopy(a[i], 0, b[i], 0 , a[i].length);
}
```